

Table of Contents

Giải thuật Di truyền Mô hình Đảo cho Bài toán Người Giao Hàng trên Cụm MPI

Môn: Lập trình song song — **Đồ án cuối kỳ** **Nhóm:** (điền tên nhóm) — 4 thành viên **Chủ đề:** Island-model Genetic Algorithm (GA) giải Travelling Salesman Problem (TSP), song song hóa bằng MPI (OpenMPI 5.0.9) trên cụm 4 máy thật nối LAN.

Mục lục

1. Đặt vấn đề
 2. Bài toán TSP và Giải thuật Di truyền tuần tự
 3. Chiến lược song song hóa (mức độ, phân rã, ánh xạ, giao tiếp, cân bằng tải, mã giả)
 4. Kiến trúc cụm MPI
 5. Kết quả thực nghiệm (đúng đắn, kích thước N, độ mịn, speedup)
 6. Khó khăn và cách khắc phục
 7. Kết luận
 8. Phụ lục: cách chạy lại
-

1. Đặt vấn đề

TSP (Travelling Salesman Problem): cho N thành phố và khoảng cách giữa chúng, tìm một lộ trình khép kín đi qua **mỗi thành phố đúng một lần** sao cho **tổng quãng đường ngắn nhất**. TSP là **NP-hard**: số lộ trình khả dĩ là $(N-1)!/2$; với $N=50$ đã là $\sim 3 \times 10^{62}$ khả năng — không thể duyệt hết. Ta dùng **metaheuristic** (Giải thuật Di truyền) để tìm nghiệm tốt trong thời gian chấp nhận được.

Vì sao song song? GA cần quần thể lớn và nhiều thế hệ $\rightarrow$ tốn thời gian. Mô hình **Đảo (Island model)** chia quần thể thành nhiều đảo chạy **đồng thời** trên nhiều máy, định kỳ **di cư** cá thể tốt giữa các đảo. Đây vừa là bài toán thú vị, vừa thể hiện kỹ thuật song song hóa thật sự (không phải chia việc tầm thường).

2. Bài toán TSP và Giải thuật Di truyền tuần tự

2.1. Biểu diễn

- **Cá thể:** một hoán vị của $0..N-1$ — thứ tự thăm các thành phố.
- **Fitness:** nghịch đảo độ dài lộ trình; tour càng ngắn càng tốt.
- **Độ dài tour:** tổng khoảng cách Euclid giữa các thành phố liên tiếp, khép kín.

2.2. Các toán tử di truyền

Toán tử	Mô tả	Vì sao chọn
Khởi tạo	Sinh ngẫu nhiên các hoán vị	Đa dạng ban đầu
Chọn lọc giải đấu	Lấy ngẫu nhiên k cá thể, giữ tốt nhất	Áp lực chọn lọc điều chỉnh qua k
Lai ghép thứ tự (OX)	Giữ đoạn của cha 1, điền phần còn lại theo thứ tự cha 2	Bảo toàn hoán vị hợp lệ
Đột biến (swap + đảo đoạn)	Đổi chỗ 2 thành phố / đảo một đoạn	Thoát cực trị cục bộ
Elitism	Giữ cá thể tốt nhất sang thế hệ sau	Nghiệm không bao giờ xấu đi

2.3. Vòng tiến hóa tuần tự (mã giả)

khởi tạo quần thể ngẫu nhiên

lặp qua từng thế hệ:

 sắp xếp quần thể theo độ dài tour

 giữ lại cá thể tinh hoa

 lặp đến khi đủ quần thể mới:

 cha1, cha2 = chọn lọc giải đấu

 con = lai ghép OX(cha1, cha2); đột biến(con)

 thêm con vào quần thể mới

 cập nhật độ dài; ghi lại tour tốt nhất

Mã nguồn: `cpp/ga_core.hpp` (GA operators), `cpp/local_search.hpp` (2-opt/Or-opt).
Solver hoàn toàn bằng C++; Python chỉ dùng để vẽ/visualize.

3. Chiến lược song song hóa

3.1. Mức độ song song & kỹ thuật phân rã

- **Mức độ song song: SONG SONG DỮ LIỆU (data parallelism).** Quần thể (dữ liệu) được chia cho nhiều tiến trình; mỗi tiến trình MPI = **một đảo** xử lý quần thể con của mình.
- **Kỹ thuật phân rã: HỖN HỢP (hybrid)** gồm:
 - **Phân rã dữ liệu (data decomposition):** tổng quần thể P chia đều P/p cá thể/đảo.
 - **Phân rã thăm dò (exploratory decomposition):** mỗi đảo là một cuộc tìm kiếm độc lập trong không gian nghiệm (seed khác nhau $\Rightarrow$ khám phá vùng khác nhau).
- **Hạt (grain):** một đảo = một tác vụ; kích thước hạt = P/p cá thể $\times$ gens thế hệ.

3.2. Ánh xạ tiến trình (mapping technique)

- **Sơ đồ ánh xạ: KHÔNG dùng vòng (ring) trao đổi láng giềng** — mỗi tiến trình MPI là một đảo độc lập, gán theo thứ tự hostfile (sequence mapping).
- **Gán: một đảo cho mỗi tiến trình (1 process / island).** Trên cụm, dùng cơ chế ánh xạ tuần tự (--map-by seq --bind-to none) để gán đảo cho từng máy theo hostfile — cần thiết vì 4 máy có topology CPU khác nhau (mapper mặc định theo topology sẽ loại bỏ máy có topology lệch với máy launcher).

3.3. Chiến lược giao tiếp & topology

- **Topology giao tiếp: KHÔNG phải vòng (ring)** — mọi đảo tham gia TẬP THỂ (collective) qua Allreduce + Bcast, không có khái niệm láng giềng trái/phải.
- **Di cư (partial global-best migration):** mỗi K thế hệ (mặc định K=20), MPI_Allreduce với MPI_MINLOC tìm ĐỒNG THỜI (a) độ dài tour ngắn nhất toàn cục và (b) rank của đảo giữ nó. Đảo thắng MPI_Bcast tour đó ra TẤT CẢ đảo khác (không chỉ láng giềng).

rank 0 rank 1 rank 2 ... rank p-1

\ | | /

\ | | /

> Allreduce(MINLOC) tìm global-best <

|

Bcast tour của owner -> mọi rank khác

|

mỗi rank khác: OX-splice 1 đoạn ngẫu nhiên của tour đó vào --migrants cá thể cục bộ (không clone toàn bộ tour)

- Sau khi nhận tour qua Bcast, MỖI đảo khác (không phải đảo thắng) chỉ splice một ĐOẠN NGẪU NHIÊN liên tục của tour đó vào --migrants cá thể cục bộ (mặc định 3) bằng OX crossover (đoạn từ global-best, phần còn lại từ một cá thể cục bộ ngẫu nhiên), thay thế các slot tệ nhất. Cách này lan truyền sub-route tốt mà KHÔNG clone toàn bộ tour $\Rightarrow$ giữ được đa dạng giữa các đảo (khác hẳn ring-migration clone toàn bộ cá thể tốt nhất, đã bị loại bỏ ở các phiên bản trước - xem §6).
- KHÔNG phải master-slave — đây là mô hình ngang hàng (peer), giao tiếp tập thể (collective), không có tiến trình điều phối trung tâm cho bước di cư.
- Gom kết quả cuối: MPI_Allreduce với MPI_MINLOC (cùng cơ chế dùng cho di cư) tìm global-best cuối cùng; đảo thắng gửi lộ trình về rank 0 bằng MPI_Send/MPI_Recv (point-to-point, chỉ 1 lần ở cuối chương trình).

3.4. Cân bằng tải (load balancing)

- **Cân bằng tĩnh, tự nhiên:** mọi đảo có cùng P/p cá thể và cùng số thế hệ $\Rightarrow$ khối lượng tính toán gần bằng nhau. Không cần lập lịch động.
- Khi phần cứng đồng nhất, độ lệch thời gian rảnh $\approx 0\%$ (đo thật: idle skew 1.8% ở 48 rank/4 máy, xem §5.3). Khi phần cứng dị thể (RAM/CPU khác nhau giữa các máy), máy nhanh phải CHỜ ở rào đồng bộ (barrier) tại điểm di cư cho máy chậm bắt kịp $\Rightarrow$ phần lớn 'comm time' đo được trên cụm thật chính là thời gian chờ này, không phải chi phí truyền dữ liệu thuần (xem §5.5).

3.5. Mã giả thuật toán SONG SONG

INPUT: cities, p (số tiến trình), G (gens), Pop (quần thể/đảo), K (chu kỳ di cư), M (số cá thể nhận di cư)

```
rank = MPI_Comm_rank ; size = MPI_Comm_size
rng = seed(BASE + rank*1000)          # mỗi đảo seed riêng
pop = [random_tour() for _ in 1..Pop]
t0 = Wtime() ; comm_time = 0
```

for gen in 1..G:

 sort pop theo độ dài tour ; new = [elite]

 while |new| < Pop:

 p1,p2 = tournament_select(pop)

 child = OX(p1,p2) ; mutate(child) ; new.append(child)

```

pop = new
if K>0 and gen%K==0 and size>1:          # DI CƯ tập thể (KHÔNG phải ring)
    t = Wtime()
    (best_len, owner) = MPI_Allreduce((local_best_len, rank), MINLOC)
    best_tour = MPI_Bcast(tour của owner, root=owner)
    comm_time += Wtime() - t
    if rank != owner:
        for i in 1..M:                    # splice 1 đoạn ngẫu nhiên, KHÔNG clone toàn bộ
            seg = random_contiguous_segment(best_tour)
            mate = random_local_individual(pop)
            pop[worst_slot_i] = OX(seg, mate)
    if patience>0 and global_best không cải thiện trong patience gens: break # mọi rank
    dừng cùng lúc

elapsed = Wtime() - t0
(global_best, win) = MPI_Allreduce((best_local, rank), MINLOC) # gom kết quả CUỐI
(giống cơ chế di cư)
nếu rank==win và win!=0: MPI_Send(tour -> 0)
rank 0 in/lưu tour ; gom (compute,comm) mọi rank để vẽ biểu đồ

```

Mã nguồn song song: cpp/tsp_island.cpp (toàn bộ giải thuật MPI - không có bản Python tương đương trên nhánh này).

3.6. Vì sao mô hình này hợp cụm mạng (LAN/WAN) độ trễ cao?

Island-GA có tỉ lệ tính/giao tiếp cao: mỗi đảo tính nặng suốt K thế hệ rồi chỉ trao đổi 1 tour (vài nghìn số nguyên tùy N) mỗi lần di cư. Tuy nhiên đo thật trên cụm (§4) cho thấy chi phí thực tế không chỉ là độ trễ mạng/băng thông thuần, mà còn do thời gian CHỜ ở rào đồng bộ khi các máy không đồng nhất - xem phân tích chi tiết ở §5.5 và §5.7.

4. Kiến trúc cụm MPI (thực tế đã dựng)

Nhóm dựng cụm 4 máy thật (laptop/desktop của các thành viên) nối cùng LAN — IP đổi vài lần trong quá trình làm (chuyển mạng/hotspot), ánh xạ tên node -> IP qua /etc/hosts, cập nhật lại mỗi lần đổi mạng.

LAN (cùng /etc/hosts trên mọi node)

node1	node2	node3	node4
Windows/WSL	Ubuntu	Windows/WSL	Windows/WSL
Ryzen 7	i5-11400H	i5-12500H	i5-12500H

4800HS (launcher)

8c/16t,7.5GB 6c/12t,23GB 8c/16t,3.7GB 8c/16t,7.7GB

- **OpenMPI 5.0.9 build từ NGUỒN (pinned tại /opt/openmpi-5.0.9) trên cả 4 máy - bắt buộc dùng đúng 1 bản build duy nhất, apt version khác nhau giữa các máy gây lỗi PMIx mismatch.**
- **SSH không mật khẩu (key-based) từ launcher (node2) tới 3 máy còn lại, KỂ CẢ loopback SSH vào chính node2 (mpirun cần SSH vào mọi host trong hostfile, kể cả host đang chạy launcher).**
- Code đồng bộ qua git (nhánh main), cùng đường dẫn ~/parallel-tsp mọi máy (dùng symlink nếu repo thật nằm ở path khác trên launcher).
- Bộ khởi chạy: cluster/run_cluster.sh (phân tán, pin launch-agent + --map-by seq) và cluster/run_local.sh (một máy).

Chi tiết dựng cụm: cluster/00_build_openmpi.sh -> 01_install.sh -> 02_ssh_setup.sh -> 03_sync_code.sh, xem README.md mục Cluster.

Lưu ý quan trọng: cả 4 máy đều dùng CPU có hyperthreading (8 lõi thật/16 luồng trên node1/3/4, 6 lõi thật/12 luồng trên node2). cluster/hosts CAP mọi node ở 12 rank (= số luồng tối đa của máy yếu nhất - node2) một cách CÓ CHỦ ĐÍCH để tránh oversubscribe hyperthreading - dùng đủ 16 rank/máy đã được đo là LÀM CHẬM ĐI (xem §5.7), không phải nhanh hơn.

5. Kết quả thực nghiệm

Đo trên cụm 4 node thật: node1 = Windows/WSL, AMD Ryzen 7 4800HS (8c/16t, 7.5GB); node2 = Ubuntu native, Intel i5-11400H (6c/12t, 23GB, launcher); node3 = Windows/WSL, Intel i5-12500H (8c/16t, 3.7GB); node4 = Windows/WSL, Intel i5-12500H (8c/16t, 7.7GB). OpenMPI 5.0.9 (build từ nguồn). Sinh số liệu bằng python/experiments.py + cluster/run_cluster.sh; biểu đồ trong results/exp_*.png.

5.1. Kiểm tra TÍNH ĐÚNG ĐẮN của nghiệm song song

- Viết script kiểm tra ĐỘC LẬP (python/validate_tour.py, không dùng lại code C++) gồm 3 mức: (1) tour là hoán vị hợp lệ của 0..N-1 - mỗi thành phố đúng 1 lần; (2) recompute độ dài tour từ tọa độ gốc, so khớp với 'Best length' solver in ra; (3) với $N \leq 10$, brute-force toàn bộ hoán vị để so với nghiệm tối ưu tuyệt đối.

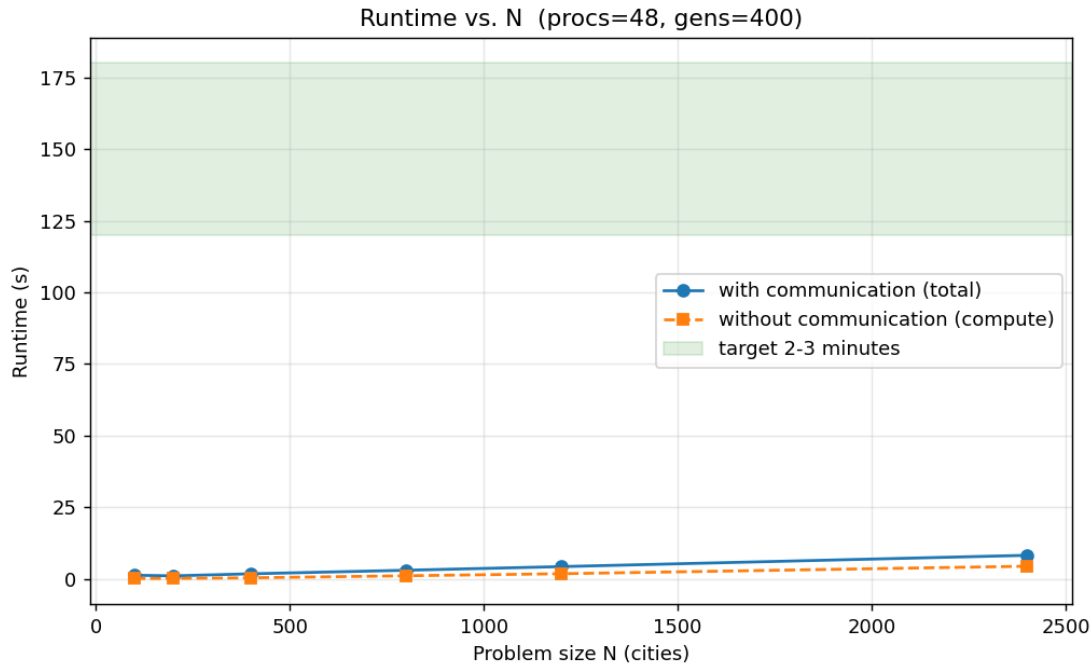
- N=8 (seed=42): route hợp lệ, recompute khớp output (240.4889 vs 240.4900), và GA TÌM ĐÚNG TỐI ƯU TUYỆT ĐỐI so với brute-force (gap 0.00%). Log thật: results/validate_n8.log.
- N=200: route hợp lệ (200/200, không trùng), recompute khớp output (2348.0052 vs 2348.0100). N quá lớn để brute-force. Log thật: results/validate_n200.log. → Solver trả về tour hợp lệ và giá trị in ra được tính đúng từ tour thật.

5.2. Tìm N* sao cho runtime $\approx$ 2-3 phút (và phát hiện trần RAM của cụm)

Phát hiện quan trọng trước khi chọn N: cpp/ga_core.hpp cấp $O(N^2)$ bộ nhớ (ma trận $N \times N$ khoảng cách) CHO MỖI rank. Với 12 rank/máy, N=6400 cần ~3.93GB/máy - vượt RAM của node3 (3.7GB) → thử trực tiếp: TREO THẬT, không xong sau 3 phút (swap thrashing, xác nhận bằng free -h). Đây là trần vật lý cứng của cụm, không phải do GA hay do mạng chậm.

N	Tổng (s)	Compute (s)	Comm (s)
1200	54.54	28.65	25.90
1800	86.99	52.66	34.33
2400	154.27	90.22	64.05

Quy trình: dò qua N (gens=400 cố định) tới N=3200 (15.43s, tăng siêu tuyến tính do cache-miss trên ma trận N^2 lớn) rồi N=6400 (timeout). Chuyển sang dò qua gens tại N=2400 (an toàn RAM) để đạt mục tiêu thời gian mà không vượt trần RAM. Đo lặp lại 3 lần ở N=2400/gens=10500/procs=48/sync=20: 159.14s, 154.27s, 145.97s (trung bình $\approx$ 153s, dao động $\pm 4.5\%$) - cả 3 đều trong mục tiêu 120-180s. Số liệu chính thức (có log đầy đủ): 145.97s, results/nstar_run.log + results/nstar_final_stats.csv. Chọn N* nhỏ (2400, không phải giá trị lớn hơn) để $2 \times N^* = 4800$ vẫn an toàn RAM cho thí nghiệm speedup ở §5.4.

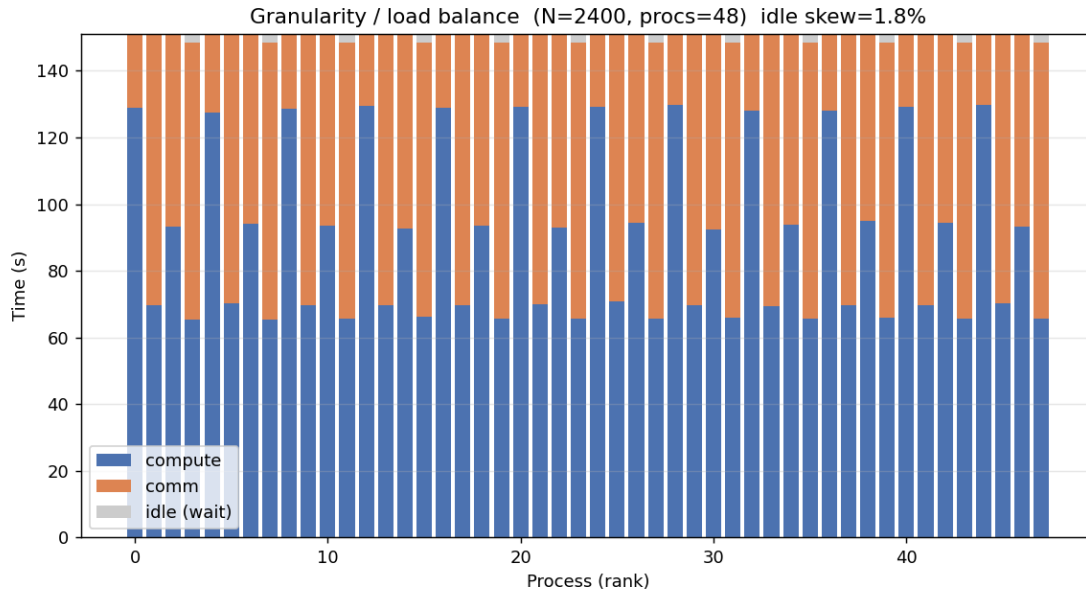


Runtime vs N tại gens=10500 cố định (procs=48, sync=20) - bảng dùng để chọn $N^=2400$*

5.3. Kiểm tra độ mịn (granularity) & cân bằng tải

Chạy tại $N^*=2400$, $gens^*=10500$, $procs=48$ (toàn bộ 4 máy, 12 rank/máy), vẽ thời gian từng rank (compute/comm/idle khác màu, cột chồng) - results/exp_gran.png, exp_gran.csv (python3 python/experiments.py gran --procs 48 --size 2400 --gens 10500 --sync 20 --hostfile cluster/hosts.seq48).

idle skew = 1.8% ($\leq 25\%$) $\Rightarrow$ hệ cân bằng tốt. Compute dao động 65-130s/rank, comm dao động ngược lại (22-83s/rank) - KHÔNG phải do rank giữ global-best tốn compute để broadcast (Bcast/Allreduce vốn là thao tác comm/chờ, không tốn thêm compute cho người gửi). Nguyên nhân hợp lý hơn: (1) phần cứng không đồng nhất giữa 4 máy (node3 RAM thấp hơn node4 dù cùng CPU); (2) GA có yếu tố ngẫu nhiên (mỗi đảo hội tụ tốc độ khác nhau tùy seed); (3) thời gian chờ đồng bộ tại mỗi điểm sync (phần lớn của 'comm_s' đo được). idle time luôn gần 0 nên dù compute/comm lệch nhau, tải vẫn được sử dụng hợp lý, không có rank rảnh thật.



Độ mịn / cân bằng tải

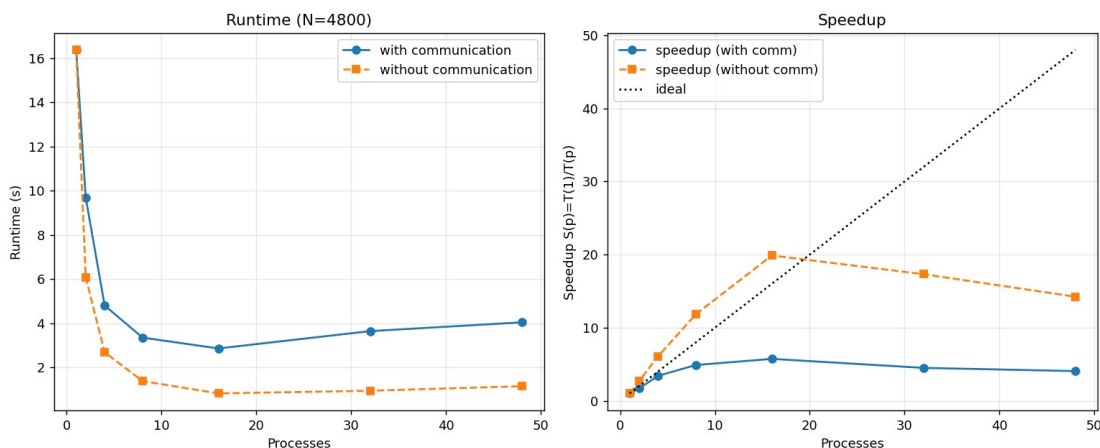
5.4. Độ tăng tốc (speedup) tại $2 \times N^* = 4800$ thành phố — có và không có thời gian truyền thông

Giữ tổng quần thể cố định = 480 (strong scaling), biến số tiến trình 1,2,4,8,16,32,48; N=4800 thành phố, gens=400, sync=20; mỗi mốc đo thời gian có và không comm (results/exp_speedup.csv, exp_speedup.png - python3 python/experiments.py speedup --procs 1 2 4 8 16 32 48 --size 4800 --gens 400 --sync 20 --hostfile cluster/hosts.seq48).

procs	Tổng (s)	Comm (s)	Speedup (có comm)	Speedup (không comm)	Efficiency
1	16.36	0.000	1.00	1.00	100%
2	9.69	3.61	1.69	2.69	84%
4	4.80	2.11	3.41	6.09	85%
8	3.35	1.97	4.89	11.86	61%
16	2.86	2.03	5.73	19.88	36%
32	3.64	2.70	4.49	17.32	14%
48	4.04	2.89	4.05	14.24	8%

procs	Tổng (s)	Comm (s)	Speedup (có comm)	Speedup (không comm)	Efficiency
-------	----------	----------	----------------------	----------------------------	------------

- Speedup tăng đều tới 16 procs, đạt ĐỈNH 5.73× (compute thuần scale 19.88× nếu bỏ comm) - không bao giờ tụt dưới 1, khác hẳn thí nghiệm ở N nhỏ (N=400) nơi speedup tụt dưới 1 khi procs>4.
- Sau 16 procs, speedup giảm nhẹ (32: 4.49×, 48: 4.05×) do comm chiếm tỷ trọng lớn hơn khi mỗi island còn ít việc - nhưng vẫn >4×. Kích thước bài toán đủ lớn khắc phục được vấn đề overhead đồng bộ quan sát thấy ở N nhỏ.



Speedup & Efficiency

5.5. Loại node3 khỏi cụm — node3 có phải 'vấn đề' không?

Giả thuyết cần kiểm chứng: node3 (RAM thấp nhất, 3.7GB) có phải là rank chậm nhất? Thử bỏ node3, chạy hết công suất 3 máy còn lại, ở đúng config N*/gens* (N=2400, gens=10500, sync=20, pop=200/island - so sánh ngang được).

Cấu hình	Ranks	Tổng (s)	Comm (s)	Compute (s)
4 node, 12 rank/máy (gốc, có node3)	48	154.27	64.05	90.22
3 node, 16/12/16	44	227.61 ❌	115.62	111.99

Cấu hình	Ranks	Tổng (s)	Comm (s)	Compute (s)
rank (bỏ node3, oversub. HT)				
3 node, 12 rank/máy (bỏ node3, cap HT đúng)	36	141.81	56.84	84.97

Lần thử đầu (SAI): dùng hết 16 luồng của node1/node4 (bỏ node3) → 44 rank, 227.61s - CHẬM HƠN baseline 48-rank có node3 (154.27s)! `lscpu` cho thấy cả 3 máy đều có hyperthreading (8 lõi thật/16 luồng) - dùng đủ 16 luồng oversubscribe hyperthreading 2× (thay vì 1.5× như cấu hình gốc cap 12), HT không cho speedup tuyến tính ⇒ phản tác dụng.

Lần thử đúng (vẫn cap 12 rank/máy, chỉ bỏ node3): 36 rank, 141.81s - NHANH HƠN baseline ~8%. Xác nhận node3 đúng là straggler nhẹ. `lscpu` cho thấy node3 có CPU CÙNG MODEL với node4 (i5-12500H) - chậm hơn nhiều khả năng do áp lực bộ nhớ kết hợp với sự không đồng nhất hệ thống (RAM 3.7GB so với 7.7GB của node4), nhưng chỉ đo 1 lần mỗi cấu hình nên chưa đủ cơ sở khẳng định tuyệt đối RAM là nguyên nhân duy nhất.

Bài học phương pháp luận: phải tách biệt 2 biến (loại node vs tăng rank/máy) khi so sánh - nhìn riêng lẻ lần thử đầu sẽ kết luận SAI 'bỏ node3 không giúp gì', trong khi thực ra là do biến nhiễu hyperthreading. Quyết định lịch sử của dự án (cap 12 rank/máy) là đúng đắn, đã được kiểm chứng lại ở đây.

5.6. So sánh --sync 0/20/100/200 (N=4800, procs=48, gens=400)

Giả thuyết: tăng --sync (đồng bộ thừa hơn) sẽ giảm comm time. Kết quả thực tế KHÁC giả thuyết:

sync	Tổng (s)	Comm (s)	Compute (s)	Best length (1 seed)
0 (baseline)	38.29	0.55	37.75	174825.61
20	42.68	20.96	21.72	175298.77

sync	Tổng (s)	Comm (s)	Compute (s)	Best length (1 seed)
100	44.57	21.23	23.34	175584.19
200	42.85	19.99	22.86	175368.18

Tăng --sync từ 20→100→200 KHÔNG làm giảm comm time (vẫn ~20-21s, sai biệt nằm trong nhiễu đo) - chỉ --sync 0 (tắt hẳn migration) mới giảm comm gần 0 (0.55s). Giải thích: với 48 rank trên 4 máy không đồng nhất, phần lớn 'comm time' là thời gian CHỜ ở rào đồng bộ để rank chậm nhất bắt kịp, không phải chi phí truyền dữ liệu của broadcast. Giảm tần suất sync không giảm tổng thời gian chờ vì lịch tiến độ chỉ dồn thành các lần chờ lâu hơn nhưng ít lần hơn. LƯU Ý QUAN TRỌNG: mỗi dòng trong bảng dưới chỉ là 1 lần chạy (1 seed) - KHÔNG nên kết luận sync nào cho chất lượng nghiệm tốt hơn từ bảng này (sai biệt Best length ~0.4% hoàn toàn có thể là nhiễu ngẫu nhiên của GA); cần ≥ 5 -10 seed/cấu hình để kết luận chắc về chất lượng - ngoài phạm vi báo cáo này.

6. Khó khăn và cách khắc phục

Khó khăn	Nguyên nhân	Cách khắc phục
IP cluster đổi 3 lần trong buổi làm (LAN -> hotspot -> LAN)	Mạng WiFi/hotspot không cố định IP	Cập nhật ~/.ssh/config + /etc/hosts mỗi lần đổi, accept-new host key
mpirun không loopback SSH vào chính launcher (node2)	~/.ssh/authorized_keys của node2 rỗng	Thêm public key của node2 vào authorized_keys của chính nó
--stats CSV rỗng khi đọc từ launcher	Rank 0 (ghi file) luôn nằm ở host ĐẦU TIÊN trong hostfile, không phải launcher	scp file --stats từ host của rank 0 về sau khi chạy (sửa trong experiments.py)
-np > 4 lỗi 'not enough node entries' với --map-	seq mapper bỏ qua hoàn toàn slots=N trong	Tạo hostfile mở rộng N dòng (round-robin theo

Khó khăn	Nguyên nhân	Cách khắc phục
by seq	hostfile, cần đúng N dòng cho N rank	node) thay vì dùng slots=
N=6400 treo thật (>3 phút không xong)	ga_core.hpp cấp $O(N^2)$ RAM/rank; 12 rank x N^2 x 8 byte vượt RAM node3 (3.7GB) -> swap thrashing	Giữ N^* nhỏ (≤ 2550) để $2 \times N^*$ vẫn an toàn RAM trên mọi node
Bỏ node3 dùng hết 16 luồng lại CHẬM HƠN (lần thử đầu)	CPU có hyperthreading (8 lõi thật/16 luồng); dùng đủ 16 oversubscribe HT 2x	Vẫn cap 12 rank/máy (=số luồng máy yếu nhất) khi bỏ node3 -> nhanh hơn ~8%
Tăng --sync không giảm comm time như kỳ vọng	Phần lớn comm time là thời gian chờ rào đồng bộ do máy không đồng nhất, không phải tần suất broadcast	Không có cách khắc phục đơn giản; cần phần cứng đồng nhất hơn hoặc chia population theo tốc độ máy
data/cities_2400.txt thiếu trên node3 -> segfault	rsync code không tự đồng bộ data file mới sinh	rsync lại file dữ liệu sang mọi node sau khi sinh
node1 lù 1 commit git so với các node khác	Chưa pull sau khi launcher cập nhật code	git pull --ff-only trên node1 trước khi build lại

7. Kết luận

- Dựng được cụm MPI 4 máy thật (LAN), build OpenMPI 5.0.9 từ nguồn, chạy Island-GA phân tán thật tới 48 rank đồng thời trên cả 4 máy.
- Cài đặt Island-GA cho TSP hoàn toàn bằng C++ (cpp/ga_core.hpp, cpp/local_search.hpp, cpp/tsp_island.cpp); Python chỉ dùng để vẽ/visualize.
- **Đúng đắn: script kiểm tra độc lập (validate_tour.py) xác nhận tour luôn là hoán vị hợp lệ, recompute khớp output; N=8 GA tìm đúng tối ưu tuyệt đối so với brute-force.**
- $N^*=2400/\text{gens}^*=10500$ cho runtime mục tiêu 2-3 phút (145.97s, lặp lại 3 lần 146-159s). Granularity tại N^* : idle skew 1.8% - cân bằng tải tốt trên

48 rank/4 máy không đồng nhất. Speedup tại $2 \times N^* = 4800$: đạt đỉnh $5.73 \times$ ở 16 procs (compute thuần scale $19.88 \times$) - không tụt dưới 1, khác hẳn N nhỏ.

- Phát hiện quan trọng: (1) trần RAM $O(N^2)$ của ga_core.hpp là giới hạn vật lý cứng của cụm không đồng nhất; (2) bỏ node3 chỉ giúp ích nếu KHÔNG vi phạm giới hạn hyperthreading; (3) tăng --sync không giảm comm time vì nút thắt là thời gian chờ đồng bộ do máy không đồng nhất, không phải băng thông/tần suất broadcast.
 - Mở rộng: chia quần thể theo tốc độ máy thay vì chia đều; chạy nhiều seed để kết luận chắc về ảnh hưởng của --sync tới chất lượng nghiệm; theo dõi cache-miss/vmstat trong lúc chạy để xác định chính xác hơn nguyên nhân straggler của node3.
-

8. Phụ lục: Cách chạy lại

Build + unit tests

```
cd cpp && make CXX=/opt/openmpi-5.0.9/bin/mpicxx  
make test
```

Chạy MỘT MÁY

```
export PATH=/opt/openmpi-5.0.9/bin:$PATH LD_LIBRARY_PATH=/opt/openmpi-5.0.9/lib  
mpirun --oversubscribe -np 4 ./cpp/tsp_island data/cities_50.txt --gens 500 --sync 20
```

Chạy PHÂN TÁN trên cụm (từ launcher)

```
bash cluster/run_cluster.sh cluster/hosts 4 ./cpp/tsp_island data/cities_50.txt --gens 500 --  
sync 20
```

Sinh số liệu + biểu đồ cho báo cáo

```
python3 python/experiments.py size --procs 48 --sizes 1200 1800 2400 --gens 10500 --  
hostfile cluster/hosts.seq48
```

```
python3 python/experiments.py gran --procs 48 --size 2400 --gens 10500 --hostfile  
cluster/hosts.seq48
```

```
python3 python/experiments.py speedup --procs 1 2 4 8 16 32 48 --size 4800 --gens 400 --  
hostfile cluster/hosts.seq48
```

Correctness validation

```
python3 python/validate_tour.py data/cities_8.txt "<route>" <best_length>
```

Toàn bộ solver là C++ (không có bản Python tương đương trên nhánh này); biên dịch bằng `make CXX=/opt/openmpi-5.0.9/bin/mpicxx` trong `cpp/`.